

JAVASCRIPT FUNDAMENTALS

FORMATIVE ASSESSMENT

KM-02: Principles of Programming with JavaScript

ELECTRONIC LEARNER ASSESSMENT PAPER

Skills Programme	SP-220373: JavaScript Programmer
Module code	900219-000-00-KM-02
NQF level / credits	Level 4 / 9 credits
Assessment type	Electronic internal formative assessment
Duration	150 minutes
Total	80 marks
Assessment decision	Competent (C) / Not Yet Competent (NYC)

Learner details

Full name	Shane Davis
ID / learner number	8309175131082
Cohort / group	Group 1
Assessment date	02 September 2026
Electronic filename	KM-02: Principles of Programming with JavaScript

How to complete and submit this assessment

- Download this file from Moodle and open it in Microsoft Word. Select Enable Editing if Word displays a protected-view message.
- Complete the yellow fields. Use the dropdowns for multiple-choice, True/False and both Mix-and-Match sets.
- Click inside every written-response field and type your answer. The fields expand as you type.
- The theoretical and scenario sections test explanation and application. Do not write JavaScript code.
- Answer independently and follow the approved assessment rules for assistance, references and academic integrity.
- Save regularly and rename the file: Surname_Name_KM02_Formative_Assessment.docx.
- Upload the completed .docx file to the correct Moodle submission link. Do not convert it to PDF.

Electronic assessment note

This assessment totals 80 marks. The assessor records the score and the final Competent (C) or Not Yet Competent (NYC) decision according to the provider-approved assessment rule. Complete every field before submission.

Assessment structure

Section	Assessment method	Marks	Electronic response method
A	Multiple-choice	10	Select one option per question.
B	True or False	10	Select T or F per statement.
C	Two Mix-and-Match sets	10	Select one letter per concept.
D	Four theoretical questions	20	Type responses in the expanding fields.
E	Two guided scenarios	30	Type responses under each guided part.
TOTAL		80	
SECTION A Multiple-choice questions			10 MARKS

Select the best answer from the dropdown below each question. Each question carries 1 mark.

1. What does JavaScript syntax describe? [1 mark]

- A. The rules for writing valid JavaScript instructions
- B. The colour scheme used by a web page
- C. The physical parts inside a computer
- D. The order in which files are uploaded to GitHub

Your answer: A

2. Which declaration is most suitable for a value that must be reassigned later? [1 mark]

- A. const
- B. let
- C. typeof
- D. return

Your answer: B

3. Which data type is most suitable for an approval status that can only be true or false? [1 mark]

- A. String
- B. Number
- C. Boolean
- D. Array

Your answer: C

4. Why is strict equality (===) recommended for beginners? [1 mark]

- A. It changes a string into a number automatically
- B. It compares only the data type
- C. It assigns a new value to a variable
- D. It compares both value and data type

Your answer: D

5. Which conditional structure is often clear when one value is compared with several exact options? [1 mark]

- A. switch
- B. while
- C. const
- D. map()

Your answer: A

6. What is the main benefit of placing repeated calculation logic inside a function? [1 mark]

- A. The calculation can never contain an error
- B. The logic can be reused and tested more easily
- C. Every variable becomes global
- D. The function automatically stores data permanently

Your answer: B

7. Which index identifies the first item in a JavaScript array? [1 mark]

- A. 1
- B. -1
- C. 0
- D. The array length

Your answer: C

8. Which loop is especially readable when processing every value in an array without needing its index? [1 mark]

- A. switch
- B. if...else
- C. while without a counter
- D. for...of

Your answer: D

9. What is one important risk of using too many global variables? [1 mark]

- A. Different parts of the program may change shared values unexpectedly
- B. Arrays can no longer store objects
- C. Functions can no longer return values
- D. The browser will always reject the script

Your answer: A

10. Which array method creates a new array containing only items that meet a condition? [1 mark]

- A. reduce()
- B. filter()
- C. push()
- D. console.log()

Your answer: B

SECTION B | True or False

10
MARKS

Select T for True or F for False. Each statement carries 1 mark.

No.	Statement	Electronic answer
1	JavaScript ignores comments when it runs the program.	T - True
2	var is the recommended default declaration for new beginner JavaScript.	F - False
3	The values "10" and 10 have the same JavaScript data type.	F - False
4	The logical AND operator (&&) requires both combined conditions to be true.	T - True
5	A final else can provide a fallback when earlier conditions are not met.	T - True
6	A returned value can be used by another part of a program.	T - True
7	An array declared with const can never have any of its items updated.	F - False
8	A while loop is safe even if nothing inside it moves the condition toward false.	F - False
9	A variable declared inside a function is normally available everywhere in the program.	F - False
10	map() creates a new array by transforming each item in an existing array.	T - True

SECTION C | Mix and Match**10
MARKS**

Complete both sets. Match every concept in Column A with the correct explanation in Column B. Each item carries 1 mark.

Set 1: Core programming concepts

Column A: Concept	Electronic answer	Column B: Explanation options
1. let	D	A. Compares both the value and the data type.
2. const	B	B. Declares a name that should not be reassigned.
3. Boolean	E	C. Chooses between several exact cases for one value.
4. Strict equality (===)	A	D. Declares a value that may be reassigned.
5. switch	C	E. Stores either true or false.

Set 2: Functions, arrays and iteration

Column A: Concept	Electronic answer	Column B: Explanation options
1. Parameter	C	A. Adds an item to the end of an array.
2. Array index	E	B. Combines array values into one final result.
3. push()	A	C. A placeholder name used in a function definition.
4. for...of	D	D. Processes each item in an iterable list without requiring the index.
5. reduce()	B	E. The position used to access an item in an array.

SECTION D | Theoretical questions**20
MARKS**

Answer all four questions in clear sentences or well-structured points. Each question carries 5 marks. No code is required.

1. Explain five practices that make beginner JavaScript easier for another person to read and maintain. Your answer must cover meaningful names, camelCase, consistent semicolons, indentation and useful comments. [5 marks]

- **Meaningful names:** Use descriptive labels that clearly state what data a variable holds.
- **camelCase formatting:** Start names with lowercase, then capitalize each new word.
- **Consistent semicolons:** Terminate every statement with a semicolon to prevent bugs.
- **Proper indentation:** Use spaces or tabs inside blocks to show structure.
- **Useful comments:** Explain the hidden purpose behind complex code logic clearly.

2. A support request contains a reference number, a requested quantity, an approval status, a list of affected devices and one structured requester record. Identify the most suitable JavaScript data type for each of the five values and briefly justify each choice. [5 marks]

- **Reference number:** String. Text type safely handles alphanumeric IDs and leading zeros.
- **Requested quantity:** Number. Numeric format enables essential counting and mathematical validation.
- **Approval status:** Boolean. True or false tracking natively handles binary states.
- **List of affected devices:** Array. Ordered collections perfectly group multiple distinct device identifiers.
- **Structured requester record:** Object. Key-value structures group related user details together cleanly.

3. Explain how a well-designed function supports reusable programming. In your answer, define a function, parameter, argument and return value, and explain why a beginner function should normally have one main job. [5 marks]

- **Function:** A named block of code built to perform a specific task.
- **Parameter:** A placeholder name defined inside a function to receive incoming data.
- **Argument:** The actual data value you pass into the function when calling it.
- **Return value:** The final data output passed back to the rest of the program.
- **One main job:** Focusing on one single task keeps code easy to test, debug, and reuse without creating unintended side effects.

4. Explain global scope, function scope and block scope. Then give two safe-variable practices that reduce accidental changes or naming conflicts in beginner JavaScript. [5 marks]

- **Global scope:** Variables declared outside any function or block, accessible everywhere in the script.
- **Function scope:** Variables declared inside a function, accessible only within that specific function.
- **Block scope:** Variables declared inside curly braces { }, restricted strictly to that specific block.
- **Default to const:** Use const for variables that should never change to prevent unintended reassignments.
- **Avoid global variables:** Keep variables inside local block or function scopes to isolate data and stop naming collisions.

SECTION E | Guided workplace scenarios**30
MARKS**

Answer every guided part. Apply KM-02 concepts in plain language. Do not write JavaScript code.

Scenario 1: Furniture quote and stock-request checker**Scenario context**

A Lewis Group furniture store wants a simple browser-based checker for sales consultants. The consultant records a quote reference, item quantity, unit price, whether the customer qualifies for a promotion, and the branch stock status. The checker must reject invalid quantities, calculate an estimated subtotal, choose a clear outcome such as Available, Back order or Invalid request, and produce a reusable customer message. The first version uses approved sample data. Answer in plain language. Do not write JavaScript code.

a. Choose and justify a suitable data type for each value: quote reference, item quantity, unit price, promotion eligibility and branch stock status. [5 marks]

- **Quote reference:** String. Text handling safely preserves alphanumeric formats, spaces, and leading zeros.
- **Item quantity:** Number. Whole integers are essential for inventory validation and mathematical multiplication.
- **Unit price:** Number. Floating-point numbers allow the system to calculate precise currency decimals.
- **Promotion eligibility:** Boolean. True/false indicators simplify the conditional logic for applying sales discounts.
- **Branch stock status:** String. Clear text labels allow the system to easily match exact categories like "In Stock" or "Out of Stock".

b. Explain the role of an arithmetic operator, a comparison operator and the logical AND operator in this checker. Give one suitable use for each. [3 marks]

- **Arithmetic operator:** Multiplication (*) calculates values mathematically by multiplying the item quantity by the unit price to find the estimated subtotal.
- **Comparison operator:** Greater-than (>) or less-than-or-equal-to (<=) checks mathematical boundaries to reject invalid quantities (e.g., stopping the process if quantity is 0 or less).
- **Logical AND operator:** The matching tool (&&) joins multiple distinct rules together, verifying that a customer is promo-eligible **and** that the branch stock status is active before applying a special discount.

c. Describe a clear conditional decision for Invalid request, Available and Back order. Explain why a fallback outcome is useful. [3 marks]

- **Invalid request:** If the item quantity is less than or equal to zero, or the unit price is negative, immediately halt the checker.
- **Available:** If the quantity is valid and the branch stock status is labeled "In Stock", approve the transaction.
- **Back order:** If the quantity is valid but the branch stock status is labeled "Out of Stock", flag the items for a delayed shipment.

d. Recommend two functions for the checker. State the one main job of each, identify suitable inputs, and explain what each function should return. [4 marks]

Calculate Order Subtotal

- **Main Job:** Handles financial math to find the total cost before tax or discounts.
- **Inputs:** Item quantity and unit price.
- **Returns:** A single calculated price number.

Determine Order Outcome

- **Main Job:** Evaluates stock status and validation rules to pick a matching status label.
- **Inputs:** Item quantity and branch stock status.
- **Returns:** A single text status string ("Invalid request", "Available", or "Back order").

Scenario 2: Delivery-request summary

Scenario context

A furniture delivery team has a list of delivery requests. Each request has a reference, suburb, priority, delivery fee and status. The team needs a summary that processes every request, counts urgent open requests, prepares readable labels, keeps only requests for a selected suburb, and calculates the total delivery fees. The solution must avoid an infinite loop and must not expose unnecessary global variables. Answer in plain language. Do not write JavaScript code.

a. Explain why an array of objects is suitable for the delivery requests. Include the purpose of the array, the objects and the named properties. [3 marks]

- **Purpose of the array:** Acts as a master list that neatly groups all deliveries together in a single, ordered collection.
- **Purpose of the objects:** Packages each separate delivery request into its own standalone container, ensuring data from different stops doesn't mix up.
- **Purpose of named properties:** Uses clear labels (like reference, suburb, priority, delivery fee, and status) to easily fetch specific details about any stop without guessing.

b. Recommend a suitable loop for processing every request. Explain how it works, how urgent open requests can be counted, and how the design prevents an infinite loop. [4 marks]

A for...of loop is the best choice because it reads straight through a list from start to finish without needing manual counters.

The Loop: It automatically pulls one delivery object out of the master array at a time, reads its data, and moves to the next one. **Counting:** You set a counter variable to 0 before the loop starts. Inside the loop, an conditional check looks at each delivery; if the priority is "Urgent" and the status is "Open", it adds 1 to your counter.

An infinite loop happens when a programming condition never reaches an end point. Because a for...of loop is hard-wired to stop automatically the exact moment it hits the final item in the array, it is physically impossible for it to run forever or crash the browser.

c. Explain how function or block scope should be used for the counter, selected suburb and temporary message. State why unnecessary globals should be avoided. [3 marks]

Selected suburb: Placed in **function scope** at the very top of your main delivery function so all loops inside that function can see the target destination.

Counter: Placed in **function scope** right before the loop starts so it can accumulate the total count across every item processed.

Temporary message: Placed inside **block scope** directly within the loop or conditional statement so it is created, used for that specific delivery, and instantly wiped from memory on the next turn.

d. Choose the best array method for each task: create a label for every request, keep only one suburb, and calculate one total delivery fee. Explain each choice. [3 marks]

1. Making labels for every request

- **The Method:** `map()`
- **How to think of it:** Imagine copying your list but changing how the information looks. If you hand `map()` a list of 10 deliveries, it will give you back a new list of 10 text labels. Your original list stays exactly the same, but you get a fresh list of formatted labels to print or read.

2. Keeping only one specific suburb

- **The Method:** `filter()`
- **How to think of it:** Think of this like a bouncer at a door. You tell `filter()` exactly which suburb you want. It checks every single delivery on your list. If a delivery is for that suburb, it lets it through. If it is for somewhere else, it leaves it out. You end up with a shorter, cleaned-up list of just the deliveries you actually need.

3. Calculating the total delivery fee

- **The Method:** `reduce()`
- **How to think of it:** Think of this as using a calculator to add up a column of numbers. `reduce()` starts at zero and looks at the delivery fee for the first item, adds it to the total, moves to the next item, adds that fee, and keeps going. In the end, it squashes the whole list down into just **one final number**—your grand total.

e. Explain why a template literal is useful for a readable delivery label and state one reason to keep modern features simple for beginners. [2 marks]

Why template literals are useful

- **Fill-in-the-blanks:** It acts like a pre-printed form.
- **No messy gluing:** You don't have to use plus signs and quotation marks to stick text together.
- **Fewer typos:** You type the sentence exactly how you want it to look, and drop data straight into placeholders like `${suburb}`.
-

Why keep features simple for beginners

- **Stops brain overload:** Too many advanced tricks confuse new learners.
- **Focuses on logic:** It lets beginners practice solving the actual problem rather than stressing over complicated rules.

Learner declaration and submission check

Learner declaration

I declare that the responses in this document are my own work, except for any approved accommodation recorded by the assessor. I understand that unauthorised assistance or copied responses may be handled under the provider assessment and academic-integrity policies.

Learner typed full name	Shane Davis
Date completed	08 September 2026
Final saved filename	KM-02: Principles of Programming with JavaScript

Assessor use only

Section A score	Assessor enters score
Section B score	Assessor enters score
Section C score	Assessor enters score
Section D score	Assessor enters score
Section E score	Assessor enters score
Total score out of 80	Assessor enters score
Overall decision	Select C or NYC ▼

Assessor feedback and remediation action

Click or tap here to type your response. The field expands as you type.